Clase 38 — La capa de enlace: servicios y entramado (framing)

Bloque: E (Capa de Enlace) · **Semana:** 13 · **Duración estimada:** 120 min **Correlativa:** Clase 37 (cierre del bloque anterior) · **Sigue con:** Clase 39 (detección de errores: paridad, checksum y CRC) **Material de referencia:** Tanenbaum cap. 3 y 4 · RFC 826 · IEEE 802.3 · IEEE 802.11 · Práctico 5 de 2008

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué la capa física no alcanza** para entregar un mensaje, y por qué sin *framing* el receptor no puede hacer nada.
2. Enumerar las **cinco funciones** de la capa de enlace y decir cuál de ellas resuelve qué problema.
3. Describir los **tres métodos de entramado** —conteo de bytes, *byte stuffing* y *bit stuffing*— y decir **por qué** cada uno existe y **qué falla** en cada uno.
4. Ejecutar un **bit stuffing bit a bit** y un **byte stuffing byte a byte**, y justificar **por qué HDLC inserta el cero después de cinco unos y no de otro número**.
5. Dibujar el formato de las **tramas de HDLC** (Información, Supervisión, No numerada) y las **cuatro fases de PPP**.
6. Resolver completo el **Ejamen 2006-12 Pregunta 2** (10 puntos).

> **Prerequisito del examen:** `2006-12 P2` (10 pts — por qué el framing, una técnica en canal orientado a bit y otra en canal orientado a byte) · `2010-03 P3` (8 pts — PPP y byte stuffing) · `2008-12 P4` y `2007-03` / `2006-12` (mismo mecanismo de transparencia, aplicado a POP3 y a un protocolo de capa 2).

PARTE 1 — TEORÍA

1.1 El problema: la capa física no sabe de mensajes

La capa física es la más tonta del modelo. Su trabajo es **convertir bits en señales** y viceversa. No sabe qué es un paquete, ni de quién es, ni a dónde va. Manda un voltaje por el cable y listo.

Y como no sabe nada de mensajes, transmite **una corriente infinita de bits**:

```
Capa física:   10110010100110101101001011010010110100101011001010110010
               ¿Dónde termina el paquete de H1?
               ¿Dónde empieza el de H2?
               NO SE SABE. No hay ningún modo de saberlo.
```

Why is this fatal? Because **el receptor necesita el mensaje entero para entregarlo a la capa 3**. Si no sabe dónde termina el datagrama que acaba de recibir, no puede calcular el checksum del datagrama, no puede quitarle la cabecera, y no puede pasárselo a IP. Y peor todavía: si el datagrama que sigue estaba pegado al anterior, **el datagrama recibido sale corrupto**, y sin embargo el receptor no tiene forma de saberlo.

```
┌─ Lo que le llega al receptor ──────────────────────────────────────┐
│  1011 0010 1001 1010 1101 0010 1101 0010 1010 1100 10 1011 0010    │
│  └── datagrama 1 ──┘└──────── datagrama 2 ──────────┘               │
│  Con el checksum de 1: NO SIRVE NADA.                              │
└────────────────────────────────────────────────────────────────────┘
```

O sea: **el entramado (*framing*) es el acto de poner marcas en el flujo de bits para que el receptor sepa dónde empieza y termina cada unidad de datos**. Sin esas marcas, la capa 2 no puede entregarle nada a la capa 3.

1.2 Las cinco funciones de la capa de enlace

La capa de enlace no hace una sola cosa: hace cinco. Y el examen las pide nombradas, así que conviene tenerlas claras **con su propósito**:

#	Función	Glosa	Pregunta que responde
1	**Entramado**	*framing*	¿dónde empieza y termina cada mensaje?
2	**Acceso al medio**	*media access control (MAC)*	¿quién tiene derecho a transmitir ahora?
3	**Detección de errores**	*error detection* (y corrección)	¿los bits que llegaron son los que salieron?
4	**Control de flujo**	*flow control*	¿el receptor va a desbordarse?
5	**Transparencia al enlace**	*data link transparency*	¿el contenido no va a romper el entramado?

El punto 5 es el que casi todos olvidan y es el que sostiene la mitad del examen de hoy. Se conecta con el punto 1 así:

```
Si el delimitador de trama es el byte 0x7E y el DATO también puede
contener 0x7E, entonces el receptor ve un final de trama donde
no la hay.

┌── Trama 1 ──┐┌── "dato" 0x7E que NO es trama ──┐┌── Trama 2 ──┐
│ 7E ... 7E   ││           7E                    ││ 7E ... 7E   │
└─────────────┘└─────────────────────────────────┘└─────────────┘
              ↑ el receptor cortó acá. Datos perdidos.
```

Transparencia significa literalmente "a través de mí podés pasar cualquier byte que quieras". Para lograrlo, hay que **escapar** (*escape*) los bytes problemáticos: cambiarlos por una secuencia que el reconocedor de tramas no confunde con un delimitador. Es la razón de existir del *stuffing*.

Sobre el control de flujo (punto 4): es el mismo concepto que en la capa de transporte (clases 19 y 20) pero aplicado a **un solo enlace**: el receptor dice "me entra(n) N tramas más" y el emisor frena. En Ethernet half-duplex el mecanismo real es colisión + *backoff* (clase 41), no un crédito explícito.

1.3 Los tres métodos de entramado

1.3.1 Conteo de bytes (*byte count*)

La idea más ingenua: **el primer byte de la trama dice cuántos bytes de datos vienen después**.

```
┌──────────┬──────────────────────────────┬──────────────┐
│ 5 (1B)   │  5 bytes de datos            │  checksum    │
└──────────┴──────────────────────────────┴──────────────┘
 longitud
```

Es elegantísimo y es una **trampa**. El problema es el acoplamiento entre la longitud declarada y los delimitadores reales:

```
EMISOR                                          RECEPTOR
  │
  │  00000101  ABCDE
  │  └─┬─┘      └─ 5 bytes
  │    └── "van 5"
  │
  │  ⚠ Si se pierde UN SOLO BIT del byte de longitud,
  │    el receptor cree que van 4 bytes (o 6).
  │    Se come un byte de DATOS como si fuera parte de la trama.
  │    Y el próximo mensaje empieza, para él, un byte tarde.
  ▼
RECEPTOR:  lee "0000101" = 5? no, lee 0000101|0 = 10 (desfasado 1)
         → DESFINICIÓN PERMANENTE.
         → lee basura como si fuera el mensaje siguiente.
```

Y lo peor no es que el error sea de esa trama: es que el error es irrecuperable y se propaga hacia adelante. El receptor quedó desfasado un byte, y **todas** las tramas siguientes se leen corrida. Sigue desfasado para siempre hasta que por casualidad coincide otra vez la alineación.

> ⚠️ **Ésta es la razón histórica por la que existe el *stuffing*.** Ningún protocolo serio usa conteo de bytes como mecanismo único de delimitación. Se usa **junto con** delimitadores (que permiten resincronizarse), nunca solo. Si el examen te pregunta "¿cuál es el problema del conteo de bytes?", la respuesta es exactamente esta: **un solo bit perdido destruye la sincronización de forma permanente, no recuperable**.

1.3.2 Byte stuffing — canal orientado a byte

Cuando el medio transmite **byte a byte** (o sea, el receptor puede mirar byte a byte y no le importa la alineación de bits), la técnica es:

1. Se elige un **byte delimitador** (*flag*) que **no pueda aparecer en los datos** por acuerdo previo. Típicamente `0x7E` (ASCII `~`).
2. Se elige un **byte de escape** (*escape*, ESC) que tampoco aparece en los datos. Típicamente `0x7D` (ASCII `}`).
3. El emisor, antes de mandar, recorre el payload: - si encuentra `FLAG` → lo reemplaza por `ESC FLAG` - si encuentra `ESC` → lo reemplaza por `ESC ESC`
4. El receptor: - ve `FLAG` → fin (o comienzo) de trama - ve `ESC` → **come el ESC** y el byte siguiente es **literal**, no marca

Ejemplo completo, byte a byte.

Datos que hay que enviar: `FLAG A B ESC C FLAG`. O sea, un payload que **deliberadamente** contiene el delimitador y el escape, para ver qué hace el emisor.

```
DATOS (payload crudo):   F  L  A  G  A  B  E  C  F  L  A  G
                          \_________/       ^  ^        \_______/
                          el FLAG de los    el ESC      otro FLAG
                          datos se escapa   se escapa

EMISOR: recorre de izquierda a derecha
──────────────────────────────────────────────────────────────
 pos  byte     ¿es delimitador/escape?      qué mando
──────────────────────────────────────────────────────────────
  1   FLAG      sí, es FLAG            →   ESC FLAG
  2   A         no                      →   A
  3   B         no                      →   B
  4   ESC       sí, es ESC             →   ESC ESC
  5   C         no                      →   C
  6   FLAG      sí, es FLAG            →   ESC FLAG
──────────────────────────────────────────────────────────────

SOBRE EL CABLE (trama completa):
    [FLAG]  ESC FLAG  A  B  ESC ESC  C  ESC FLAG  [FLAG]
      |                                      |
  inicio de trama                        fin de trama

Longitud: 1 + 2 + 1 + 1 + 2 + 1 + 2 + 1 = 11 bytes
(el payload original tenía 10)
```

RECEPTOR: desescapa

```
[FLAG]  ESC FLAG  A  B  ESC ESC  C  ESC FLAG  [FLAG]
   |       |      |  |     |    |     |  |       |
   |       └──────┘──┘     |    |     └──┘───────┘
   |       leo ESC: "el    |    |        leo ESC: "el
   |       siguiente es    |    |        siguiente es
   |       LITERAL"        |    |        LITERAL"
   ▼       ▼               ▼    ▼        ▼       ▼
inicio  F  L  A   G  A  B  E  S  C    F  L  A  G  fin
                           ^     ^
                           los dos ESC se comen UNO SOLO
                           (queda un ESC literal)

DATOS RECUPERADOS:  F L A G A B E S C F L A G      ← idénticos al original
```

El detalle fino: `ESC ESC` se colapsa a **un solo** `ESC` literal. Si colapsáramos a cero, el `ESC` original se perdería; si no colapsáramos nada, quedaría ambiguo.

PPP usa exactamente este esquema, y es lo que se pregunta en `2010-03 P3b`. HDLC también, pero con stuffing de **bits**.

1.3.3 Bit stuffing — canal orientado a bit

En un **canal orientado a bit** (*bit-oriented channel*) el receptor no tiene obligación de recibir bytes alineados: puede perder un bit y desfasarse. Por eso el HDLC no puede usar un byte delimitador, porque un byte desalineado ya no es un byte.

La regla de HDLC:

> El emisor inserta un `0` **después de cada secuencia de cinco** `1` **seguidos** en el campo de datos (descontando los flags). El receptor, al ver cinco `1` seguidos, mira el bit siguiente: si es `0`, lo descarta; si es `1`, es que venía un flag.

El flag de HDLC es `01111110` (0x7E, igual byte que en el byte stuffing, pero ahora como patrón de bits).

Ejemplo completo, bit a bit.

Datos: `0111111111110` (un 0, once unos, un 0 — es decir, once unos seguidos).

```
DATO ORIGINAL (13 bits)
 0 1 1 1 1 1 1 1 1 1 1 1 0
   └──────── 11 unos ────────┘

Paso: el emisor cuenta los unos. Al llegar al QUINTO uno, inserta un 0.

 0  1 1 1 1 1  0  1 1 1 1 1  0  1  0
 |  └── 5 ──┘     └── 5 ──┘
 1 solo            + 1 solo que ya venía
 (previo)

DATO RELLENADO (15 bits)  =  011111011111010
 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0
```

Sobre el cable:

```
[FLAG] 011111011111010  [FLAG]
01111110                01111110
└ inicio ┘              └ fin ┘
```

Recepción:

```
Leo 011111011111010
   ^^^^^ = cinco unos → el siguiente bit DEBE ser un 0 stuffed → lo como
   (no cuenta para el conteo de cinco)
Sigo: 11111 → siguiente bit es 0 → lo como
Sigo: 1 → queda uno suelto
Sigo: 0
──────────────────────────
RECUPERADO: 0 1 1 1 1 1 1 1 1 1 1 1 0  =  el original
```

¿Por qué cinco unos y no tres, ni seis, ni siete?

Esta es **la pregunta conceptual** de la clase, y es la que más puntos da.

Mirá el flag: `01111110`.

```
    0 1 1 1 1 1 1 0
      └─ SEIS unos ─┘
```

El flag tiene SEIS unos seguidos. Ese es el número clave. Ahora, ¿qué promesa tiene que hacer el relleno?

> **Promesa del relleno:** en el campo de datos rellenado, **nunca** aparecen seis unos seguidos.

Si esa promesa se cumple, entonces cualquier secuencia `01111110` que encuentre el receptor **es un flag y solo un flag**, porque en los datos es imposible. Y entonces el receptor puede buscar esa secuencia **bit a bit**, como un nilaija ciego, y **siempre se engancha en un flag verdadero**.

Ahora, ¿qué valor de relleno cumple la promesa?

Relleno cada	¿Los datos pueden tener 6 unos seguidos?	¿El flag `01111110` es único?	Overhead
cada **3** unos	no (máx. 3)	sí	**alto**: 1 bit de cada 3
cada **4** unos	no (máx. 4)	sí	medio
cada 5 unos	no (máx. 5)	**sí**	**mínimo aceptable**
cada **6** unos	**SÍ** (puede haber 6)	**NO**	—

- Con 6: los DATOS pueden contener "01111110" sin stuffing.
  0 1 1 1 1 1 1 0 ← esto es un FLAG... o son datos. INDISTINGUIBLE.
  El receptor corta la trama donde no debe. Se pierden datos.

- Con 5: los datos NUNCA tienen 6 unos. El flag es único.
  Es el MAYOR número de unos que todavía garantiza la unicidad.
  Por eso 5: es el valor ÓPTIMO — unicidad garantizada con el mínimo
  overhead posible.

Y el "slip de un bit" (la sincronización se pierde, pero se recupera). Supongamos que en el medio **se pierde un bit** (o se duplica). El receptor, a partir de ahí, lee todos los bits corrido un lugar. ¿Y ahora?

```
ANTES del error:      el receptor está alineado, ve los flags.
──────── error: se pierde 1 bit ────────
DESPUÉS:              el receptor lee corrida. Ya no reconoce el flag
                      de cierre de la trama anterior.

PERO: sigue escaneando bit a bit buscando "01111110", y como los datos
      rellenados NO pueden contener seis unos seguidos, no se va a enganchar
      con ningún falso flag.

RESULTADO: se engancha en el PRIMER flag verdadero que viene
           (el de inicio de la trama siguiente, a lo sumo una trama después).
           Se resincroniza solo. ✓
```

Con tres o cuatro, la promesa también se cumple, así que técnicamente también funciona... **pero** el costo es el doble de overhead sin ninguna ventaja. Y con stuffing cada 4, el patrón `0111 0` puede aparecer mil veces en los datos y un receptor mal implementado puede engancharse ahí. Lo que aprender de la tabla de arriba es el **principio**: *el número de stuffing debe ser estrictamente menor que la cantidad de bits iguales consecutivos del delimitador*, y se elige el mayor valor que cumple esa condición porque cada bit de relleno es overhead.

1.3.4 Framing por flags y el caso de los flags "abreviados" de 802.11

Un cuarto caso, muy usado en la práctica: **delimitar sólo con flags de principio y de fin**, y confiar en que el medio no inserte bits (canales síncronos, enlaces por fibra, enlaces con reloj compartido).

En **802.11** el entramado es distinto: no hay flags. El principio de cada trama lo marca el **preámbulo de 8 bytes** (7 de sincronismo + SFD), que es un patrón inequívoco, y el final lo marca el **FCS de 4 bytes** seguido de silencio del medio. Pero 802.11 tiene una sutileza que vale la pena conocer:

Tramas de gestión sin FCS. El estándar IEEE 802.11 permite que ciertas tramas de gestión —**Beacon, Probe Response y Association Response**— se transmitan **sin el campo de verificación de trama**, porque la estación que las recibe ya está sincronizada por reloj con el AP (se announcement cada 100 ms con beacons) y porque el riesgo de colisión en ese instante es muy bajo.

```
Trama normal:
┌────┬─────┬──────────┬──────────┬──────────┬─────┐
│FC  │Dur  │Dir/Orig  │Dir/Dest  │  cuerpo  │ FCS │
└────┴─────┴──────────┴──────────┴──────────┴─────┘
 ▲                                ▲
 │                                └─ el FCS marca el FINAL
 └─ el FC (frame control) + duración marcan el INICIO

Trama de gestión abreviada (Beacon / Probe Response):
┌────┬─────┬──────────┬──────────┬──────────┐
│FC  │Dur  │Dir/Orig  │Dir/Dest  │  cuerpo  │      ← sin FCS
└────┴─────┴──────────┴──────────┴──────────┘

                                y el SSID puede omitirse
                                (ya se conoce del beacon anterior)
```

El punto conceptual: en el caso abreviado, **el final de la trama ya no lo marca un patrón de bits sino el silencio del medio**, y el reloj de sincronización es lo que permite al receptor saber cuánto debía durar. Eso es justo lo contrario a "el flag marca el final", y por eso es un caso aparte.

1.4 HDLC — High-Level Data Link Control

HDLC es **el** protocolo de la capa de enlace con más influencia histórica: de él descienden el SDLC de IBM, el LAPB, el **PPP** e incluso el **LAPD** (token ring) y el **LLC** de IEEE 802.2, que es la parte de IEEE 802 que define la capa 2 "única" para las LAN de 802 (el resto —MAC, CSMA/CD— es específico de Ethernet).

Formato de la trama:

```
 Flag      Address   Control   Information              FCS      Flag
01111110   8 bits    8/16 b    variable (0..N)          16 b    01111110
   │          │         │            │                   │         │
   │          │         │            │                   │         └─ fin
   │          │         │            │                   └─ CRC-CCITT
   │          │         │            └─ payload (puede llevar bit stuffing)
   │          │         └─ depende del TIPO de trama (ver abajo)
   │          └─ 0xFE (undirected) | 0x01 (stations) | ...
   └─ início de trama
```

Los tres tipos de trama, que es lo que se pregunta:

Tipo	Nombre	Campo *Control*	Para qué
I	**Información**	`0xxx` (N(S), N(R), P/F)	Transporta datos **y** el piggybacking del ACK/NAK
S	**Supervisión**	`10xx` (SS, P/F)	Solo control: **RR** (00), **RNR** (01), **REJ** (10), **SREJ** (11)
U	**No numerada**	`11xx`	SET, SABM, DISC, UA, FRMR, SABME, UI

```
Trama I (Información):    [N(S)] [N(R)] [P/F]
                          └seq   └ACK   └pide ACK (poll) o fin
```

Lo importante conceptualmente: HDLC implementa un **ARQ con ventana deslizante** y **piggybacking** (el ACK viaja dentro de una trama de datos en el sentido contrario). Es el mismo esqueleto que vas a ver en la clase 40 (ARQ) y en TCP.

1.5 PPP — Point-to-Point Protocol

¿Por qué existe? El mundo de los enlaces punto a punto de 1990 (líneas telefónicas, líneas dedicadas, RS-232, enlaces conmutados de Frame Relay) necesitaba un protocolo que hiciera:

1. **Multiplexar** varios protocolos de red (IP, IPX, AppleTalk) sobre un mismo enlace. Sin esto, cada uno necesitaría su propio cable.
2. **Configurar el enlace**: negociar Quality of Service, autenticar, asignar dirección IP.
3. **Detectar errores** en el enlace (FCS), **encapsular** y ser **fiable** opcionalmente.
4. Ser **independiente del medio**: el mismo protocolo sobre cobre, fibra o línea telefónica.

Formato de la trama PPP:

```
[FLAG] Address Control Protocol    Payload    FCS        [FLAG]
7E     FF       03      2 bytes    0..1500   16 bits    7E
                │       │
                │       └─ 0021 = IPv4 | 0021...
                └─ 0x03 = "sin numerar, sin ACK, la respuesta por defecto"

Para las tramas de la capa de red se usa ESCAPE byte-stuffing: 0x7E → 0x7D 0x5E
```

Las cuatro fases de PPP (esto es lo que se pregunta en `2010-03 P3a`):

```
FASE 1 ── LCP (Link Control Protocol)
│   Configura y testea el enlace. Opciones negociables:
│   - MRU (Maximum Receive Unit)
│   - MRF (Magic Number, detectar bucles y enlaces loops)
│   - Quality of Service
│   - Autenticación (requiere que se habilite)
│   - Compression (MPPC)
▼   Secuencia: LCP-CONFIG-REQUEST → LCP-CONFIG-ACK / LCP-CONFIG-NACK
▼   Después: LCP-TERMINATE-REQUEST → LCP-TERMINATE-ACK
│
FASE 2 ── AUTENTICACIÓN
│   Sólo si LCP la habilitó. Opciones:
│   - PAP (Password Authentication Protocol): TEXTO PLANO. Inseguro.
│   - CHAP (Challenge Handshake): desafío-respuesta con MD5. Mejor.
│   Secuencia: AUTHENTICATE-REQUEST → AUTHENTICATE-CHALLENGE
│               → AUTHENTICATE-RESPONSE → AUTHENTICATE-SUCCESS/FAIL
▼
FASE 3 ── NCP (Network Control Protocol)
│   Uno por cada protocolo de capa de red que se quiera usar:
│   IPCP para IP, IPXCP para IPX, etc. Asigna la dirección IP del enlace
│   (funciona como un mini-DHCP).
▼
FASE 4 ── DATOS (carry payload)
    Tramas de datos de capa 3 encapsuladas, con la última trama en la
    cabecera PPP. State = "Network phase".
```

La respuesta oficial a `2010-03 P3a` es corta y muy precisa: lo que hace a PPP **más simple que un protocolo de acceso al medio como CSMA** es que **no opera en un medio de difusión (*broadcast*), donde muchos emisores compiten por el canal**. En un enlace punto a punto hay **exactamente dos estaciones y sólo una puede hablar**: no hay.CONTENCIÓN, y por eso **no hace falta direccionamiento MAC**. Ese es el argumento central, no "porque tiene menos campos".

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - La existencia conceptual del **entramado**: la capa física no sabe de mensajes, la capa 2 tiene que poner las marcas. Es la razón de ser de la capa. - **Bit stuffing cada cinco unos** en HDLC, con el flag `01111110`. La justificación (seis unos en el flag → cinco es el máximo que da unicidad) no cambió en 60 años. - **Byte stuffing** con `FLAG` / `ESC` en PPP, HDLC y en la práctica mil protocols. - El **byte stuffing con DLE STX/DLE ETX** como el mecanismo de transparencia de POP3 (`2008-12 P4`). Es literalmente el mismo algoritmo con otros nombres.

Cambió — el dónde, más que el cómo: - **El entramado por stuffing quedó atrás como problema de ingeniería real.** En 2026 los enlaces de datos de las LAN son de 1 a 400 Gbit/s sobre fibra o cobre de calidad, con BER del orden de `10^-18` o mejor. **Un diseño nuevo no elige stuffing**: elige un identificador de longitud explícito y un FCS fuerte (CRC-32 o mejor). El stuffing sobrevive, pero como **herencia histórica en protocolos de 1981**, no como decisión de diseño de hoy. - **El estándar de Ethernet nunca hizo stuffing.** Usa **longitud explícita** (campo *Type/Length*, y el límite de 1518 bytes) más el hecho de que el *preámbulo* de 8 bytes re-sincroniza a nivel físico en cada trama. El problema del desfasamiento de 1 bit que rompe el conteo de bytes **no existe en Ethernet**: es justamente lo que motivó el diseño con preámbulo. - **802.1Q (VLAN) insertó un tag de 4 bytes dentro de la trama** para agregar información de VLAN sin tocar el formato base. Es el ejemplo vivo de que la trama de Ethernet sigue siendo un contenedor del que se pueden meter cosas adentro. Ver §8 de `ACTUALIZACIONES.md` y la clase 42. - **PPP sigue vivo y multiplicado**: PPPoE (`RFC 2516` , verificar), PPPoA, y PPP en celdas 4G/5G — que es donde realmente se usa hoy: el "demux" del módem es PPP con IPHC. - **`RFC 826` — ARP (1982)** es el ejemplo canónico de por qué el entramado sigue siendo indispensable: un ARP request y un ARP reply comparten el mismo formato de trama y sólo el campo *Type* los distingue.

Consecuencia para el examen: la teoría del framing **se sigue preguntando tal cual** (es `2006-12 P2` , de 10 puntos). Lo que se actualiza es el segundo nivel del razonamiento: "esto se hacía porque los medios eran ruidosos y los protocolos seriales tenían que resincronizarse" es la explicación **de 2011**; "esto se hace porque el medio no garantiza alineación de bits" es la explicación **correcta**, y sigue siendo la que se corrige.

PARTE 2 — PRÁCTICA RESUELTA

> ⚠️ **Aclaración sobre el material oficial.** El **Práctico 5 de 2008** no tiene un ejercicio de entramado: sus ejercicios 1–12 son de paridad, checksum, CRC, CDMA, ALOHA, ARP y CSMA/CD. (El "Ejercicio 12" es de ARP, no de stuffing; las figuras 5.11 y 5.12 que se le atribuyen son de CDMA.) Por eso los cuatro ejercicios de esta parte son **inventados y resueltos** con el resultado **byte a byte**, que es lo que la cátedra exige.

2.1 Ejercicio 1 — Byte stuffing completo con bytes hexadecimales

Enunciado. Se transmite un payload que, en hexadecimal, es `7E 41 7D 42 7E`. El delimitador es `0x7E` y el byte de escape es `0x7D`. Determinar byte a byte qué va por el cable y qué reconstruye el receptor.

Resolución.

```
Original:  7E  41  7D  42  7E
           |   |   |   |   |
           |   |   |   |   └─ 4° tipo: aparece FLAG → hay que escapar
           |   |   |   └──── dato normal
           |   |   └──────── 3° tipo: aparece ESC  → hay que escapar
           |   └──────────── dato normal
           └──────────────── 2° tipo: aparece FLAG → hay que escapar
```

Paso 1 — el emisor recorre de izquierda a derecha y aplica dos reglas.

```
byte  entrada   regla                        salida
────  ──────    ──────────────────────────   ────────
1     0x7E      es FLAG  → ESC + original    0x7D 0x7E
2     0x41      dato normal                  0x41
3     0x7D      es ESC   → ESC + original    0x7D 0x7D
4     0x42      dato normal                  0x42
5     0x7E      es FLAG  → ESC + original    0x7D 0x7E

TOTAL sobre el cable (payload):  8 bytes
(el original tenía 5; se agregaron 3 bytes de escape)
```

```
TRAMA COMPLETA:
┌──────┬──────────────────────────────┬──────┐
│ 7E   │ 7D 7E 41 7D 7D 42 7D 7E      │ 7E   │
│FLAG  │      payload escapado        │ FLAG │
└──────┴──────────────────────────────┴──────┘
```

Paso 2 — el receptor desescapa.

```
lee  7E   → "arrancó una trama"
lee  7D   → "¡escape!"  → el próximo byte es LITERAL
lee  7E   → emite 0x7E, sigue
lee  41   → emite 0x41, sigue
lee  7D   → "¡escape!"  → el próximo byte es LITERAL
lee  7D   → emite 0x7D, sigue          ←— OJO: come UNO SOLO
lee  42   → emite 0x42, sigue
lee  7D   → "¡escape!"  → el siguiente es LITERAL
lee  7E   → emite 0x7E, sigue
lee  7E   → "terminó la trama" (FLAG suelto)

PAYLOAD RECUPERADO:  7E 41 7D 42 7E    ✓ idéntico al original
```

Interpretación. El receptor **nunca vio un `7E` dentro del payload**, aunque el payload tuviera dos. Por eso puede usar `7E` como delimitador sin ambigüedad: la propiedad que el stuffing garantiza es exactamente "*ningún `7E` en los datos del campo de datos puede ser tomado como fin de trama*".

⚠️ Errores típicos

- **Escapar dos veces** el mismo ESC (mandar `7D 7D 7D`). El emisor sólo **duplica**: cada byte que sea ESC se reemplaza por **exactamente dos**.
- **Olvidar el ESC delante de un FLAG que aparece en el medio del payload.** Es el error que arruina la trama entera.
- En el receptor, **no consumir** el byte que sigue al ESC. El ESC **no se emite**: se descarta.

2.2 Ejercicio 2 — Bit stuffing bit a bit

Enunciado. Con el protocolo HDLC (flag `01111110`), enviar el campo de datos `110110111111101`. Mostrar el resultado bit a bit.

Resolución — paso a paso, contando los unos de a cinco.

```
DATO ORIGINAL:  1 1 0 1 1 0 1 1 1 1 1 1 1 0 1
                └2┘ └2┘ └──────  7 unos  ──────┘ └┘

Recorrido, agrupando cada racha de unos:
 racha 1:  1 1            (2 unos)  → sin stuffing
 racha 2:  1 1            (2 unos)  → sin stuffing
 racha 3:  1 1 1 1 1 | 1 1    (7 unos) → se parte en 5 + 2

racha de 7 unos =  [1 1 1 1 1] (se completa el grupo) + [0 stuffed] + [1 1]
```

```
RESULTADO BIT A BIT:
 ┌─┐ ┌─┐ ┌─┬─┬─┬─┬─┐   ┌─┬─┐
 1 1 0 1 1 0 1 1 1 1 1 0   1 1 0 1
           └─── 5 unos ──┘  ↑
                            stuffed

HEXADECIMAL:  1101 1011 1111 101   →   D B F (queda 1 bit de la última fila)

En binario plano:  1101101111110101     ← 16 bits

Conteo verificado:
  original            : 15 bits
  stuffing: +1 cero   = 16 bits ✓
  ¿quedó algún grupo de 6 unos?  NO: los grupos son 2, 2, 5, 2. Máximo 5 ✓
```

Trama completa:

```
01111110   1101101111110101   01111110
[FLAG]        16 bits          [FLAG]
```

Interpretación. El receptor, al barrer, encuentra `11111` en la posición 7, come el `0` siguiente, y **el siguiente grupo de unos empieza en el bit 9**: `1 1 0 1`. Cuando llegue al flag de cierre `01111110` ve **seis** unos seguidos — imposible en datos rellenados — y lo reconoce como flag.

> **Comprobación de la regla:** el receptor sólo puede buscar el flag con seguridad **porque** los datos rellenados no tienen seis unos seguidos. Si hubiéramos stuffing cada 6, el dato `110110111111 1` (con 7 unos) se habría enviado **sin** stuffing y contendría el flag. Ver §1.3.3.

2.3 Ejercicio 3 — Conteo de bytes: qué pasa si se pierde un bit

Enunciado. Un protocolo usa conteo de bytes. Se envía el mensaje `HELLO` precedido de un byte de longitud. Explicá y **mostrá numéricamente** qué pasa en el receptor si se pierde **un solo bit** de la línea.

Resolución.

Sin errores:

```
EMISOR                                          RECEPTOR
  │                                               │
  │  00000101  H  E  L  L  O                      │
  │  └── 5 ──┘ └──── 5 bytes ────┘                 │
  │                                               │
  │                          lee 00000101 = 5     │
  │                          come 5 bytes → HELLO ✓
  ▼
```

Con un bit perdido (se borra el primer bit del byte de longitud):

```
Se transmite:      00000101  H  E  L  L  O
Se PIERDE el bit:        (borrado)      ↓ queda:  0000101...
Sobre el cable:  0 0 0 0 1 0 1  H  E  L  L  O
                  └─────┬─────┘
                el receptor lee 8 bits:  0 0 0 0 1 0 1 H

Longitud leída = 0000101H
   con H = 0 ('H' = 0x48 = 01001000, su primer bit es 0):
      00001010 = 10 bytes  ← ¡deberían ser 5!
```

```
El receptor cree que la trama tiene 10 bytes. Entonces come:

  long=0x48  H    E    L    L    O    [primer byte de la TRAMA SIGUIENTE]
    └──── 10 bytes "de datos" ──────────────────────────┘

RECUPERADO: "HELLO" + basura.  ✗  Y la trama SIGUIENTE empieza un byte tarde.
```

Y lo grave, que es la segunda parte de la respuesta:

```
estado real del cable:   [largo=5][ABCDE] [largo=3][XYZ] [largo=2][AB]...
lo que el receptor cree:  [largo=10]──────────────┘  desfasado +1 byte

→ sigue corrido 1 byte para siempre.
→ TODAS las tramas siguientes se leen mal.
→ No hay forma de resincronizar: el conteo no se autocorrige.
```

Interpretación — la frase para el examen:

> El conteo de bytes **acopla la longitud declarada con la alineación de la trama**. Un único error de bit desplaza el punto de lectura de forma **permanente e irrecuperable**: el receptor interpreta bytes de datos como longitud, y de ahí en adelante lee todas las tramas corrido. El stuffing existe justamente para que el receptor **pueda volver a engancharse** con un delimitador inequívoco después de cualquier error de alineación.

2.4 Ejercicio 4 — Flags abreviados de 802.11

Enunciado. En una red 802.11 en modo infraestructura, un Access Point difunde **Beacon** sin campo FCS y con el SSID omitido. (a) ¿Cómo sabe la estación receptora dónde termina la trama? (b) ¿Qué se rompe si hay una colisión justo durante ese beacon?

Resolución.

(a) El final **no lo marca un patrón de bits**. La estación lo deduce de:

1. El reloj de sincronización. Todas las tramas de una BSS tienen longitud CONOCIDA de antemano para cada tipo de trama: la longitud de un beacon con parámetros fijos es una constante del estándar. Entonces:

   t_inicio + longitud_conocida = t_fin

2. El silencio del medio. Al terminar la trama, el canal queda en silencio durante el Slot Time (16 µs en 802.11b) → se detecta el fin.

3. El SFD + sincronía de la trama SIGUIENTE. Si llega otra trama, el preámbulo marca el nuevo comienzo y el receptor resincroniza.

(b) El riesgo real: si el beacon se pierde o se corrompe sin que el FCS lo diga, **la estación no se entera**. Consecuencias:

- La estación cuenta el número de beacons perdidos para medir la calidad del enlace (función de RSSI y roaming). Perder beacons sin avisar degrada esa estimación.
- La sincronización temporal (TWT / timetable de Wi-Fi 6, clase 43) puede desfasarse si falta un beacon.
- El SSID omitido significa que un beacon con "hidden SSID" no se puede usar para descubrir la red: hace falta un probe activo.

Interpretación. El tradeoff es explícito: **se ahorra overhead** (4 bytes de FCS + el elemento SSID ≈ 34 bytes) **a cambio de perder la verificación de integridad**. Sólo es aceptable en tramas cuya **pérdida no es crítica** (sincronización) y cuya **retransmisión sería inútil** porque el contenido se va a repetir igual en el beacon siguiente.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2006-12, Pregunta 2 (10 puntos) — framing

> **Fuente:** Examen `eirc0612` / `sirc0612` — diciembre de 2006. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0612.pdf

Enunciado (texto del examen)

> Pregunta 2 (10 puntos)
>
> En capa 2 se introduce el concepto de "framing" o entramado. Explique por qué este proceso es necesario y describa: a) una técnica para realizar "framing" en un canal orientado a bit b) una técnica para realizar "framing" en un canal orientado a byte

Respuesta oficial (del PDF de soluciones)

> **Framing (Entramado):** - El framing le permite al DLC (Data Link Control) receptor, determinar cuando el frame comienza y termina. - Adicionalmente busca la sincronización en caso de tiempos muertos de transmisión
>
> **Framing orientado a bits.** - En este caso se utilizará un flag para separar frames. - El flag es un conjunto de bits conocidos, y para permitir la transparencia de la información se plantea el método de bit stuffing, que es la inserción de bits. - Ejemplo: considero la flag `01111110`, esto implica que la llegada de 5 1's seguidos se debe insertar un 0.
>
> **Framing orientado a bytes.** - Bytes stuffing - Para implementar el modo transparente, se utiliza el siguiente procedimiento: se utiliza otro carácter de control DLE (Data Link Escape), de la siguiente forma. - El DLE es insertado previamente a el carácter STX o ETX - Ante la presencia en el frame de un DLE, el protocolo inserta uno más. - Garantiza transparencia de datos enviados, ya que si en los datos del frame envío DLE STX, se codificará como DLE DLE STX

Análisis de la solución oficial, punto por punto

Lo que está bien y es el corazón de la respuesta:

1. **"El framing le permite al DLC receptor determinar cuándo el frame comienza y termina."** Esa es la definición, y es exactamente el punto que abre la clase.
2. **El segundo punto es fino y suma:** "*adicionalmente busca la sincronización en caso de tiempos muertos de transmisión*". O sea: el entramado no sólo delimita, también **permite resincronizar** después de un silencio en el medio (una pausa larga, un enlace que estuvo caído y volvió). Sin flag, una reanudación

del medio dejaría al receptor desalineado para siempre. **Esta es la mitad de la respuesta que la mayoría de los alumnos no da.**

3. **La transparentización del flag con bit stuffing** es la respuesta de (a).
4. **El esquema DLE con STX/ETX** es un caso particular de byte stuffing realmente usado (en SDLC, en POP3, en el protocolo de spicy de FING).

Lo que se puede mejorar — y el ejercicio para vos:

1. **La solución oficial NO justifica por qué cinco unos y no otro número.** Es lo que le falta al ítem (a). La respuesta completa es la de §1.3.3: el flag `01111110` contiene **seis** unos, y cinco es el **mayor** número de unos que garantiza que los datos rellenados no puedan contener un flag.
2. **"El DLE es insertado previamente al carácter STX o ETX"** suena a que el DLE va siempre delante de STX/ETX. Lo correcto es más general: el DLE se inserta **antes de cualquier byte de control con significado estructural** (STX, ETX, el propio DLE y, en PPP, el FLAG `0x7E`).
3. **No menciona el problema del conteo de bytes**, que es lo que da la justificación de por qué hace falta stuffing. La respuesta oficial da la solución sin dar el problema.

Mi resolución, completa

(Intro) Por qué es necesario el framing. La capa física transmite un flujo continuo de bits y **no tiene ninguna noción de mensaje**: no sabe dónde termina un paquete ni dónde empieza el siguiente. El receptor, sin embargo, necesita el mensaje **completo y alineado** para poder verificar su checksum, extraer la carga útil y entregarla a la capa de red. Si el receptor no puede ubicar los límites, cualquier paquete que le llegue junto con el siguiente estará corrupto y **no habrá forma de saberlo**. Por eso la capa 2 introduce marcas: el **entramado**. Su segundo propósito, menos obvio, es permitir la **resincronización** después de un silencio en el medio o de un error de alineación.

(a) Canal orientado a bit — HDLC: flag `01111110` + bit stuffing.

```
REGLA: el emisor inserta un 0 después de CADA cinco unos seguidos en los datos.
El flag es 01111110, que contiene SEIS unos seguidos.
∴ en los datos rellenados nunca hay seis unos
∴ 01111110 sólo puede ser un flag
∴ el receptor busca ese patrón bit a bit y siempre se engancha al correcto.
```

Ejemplo del examen, reconstruido:

```
Datos (sin rellenar):
 0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 1 1 1 1 1 1 0
   └─ 7 unos ─┘             └──────── 11 unos ────────┘

Rellenado (se insertan 3 ceros):
 0 1 1 1 1 1 0 1 1 0 1 0 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0
       ^             ^                 ^
     0 stuffed   0 stuffed         0 stuffed

Sobre el cable:
[01111110] 01111101111001000111111011111010 [01111110]
    flag            datos rellenados               flag
```

> **Nota sobre la solución oficial:** el PDF dibuja los datos **sin rellenar** y marca arriba los tres ceros a insertar. Eso es correcto como ilustración, pero si el examen pide el resultado final hay que **escribir la secuencia rellenada completa**, que es lo que hice arriba.

(b) Canal orientado a byte — byte stuffing con DLE (o ESC).

```
REGLA: se elige un byte delimitador que no aparece en los datos.
       Si aparece en el payload, se inserta antes un byte de escape (DLE/ESC).
       Si el propio escape aparece, se duplica.

Receptor: ve ESC → descarta el ESC y toma el byte siguiente como LITERAL.
```

Ejemplo con el esquema del enunciado (STX … DLE STX … ETX):

```
Payload crudo:   A B DLE STX C ETX

Emisor:
  A       → A
  B       → B
  DLE     → DLE DLE      (se duplica)
  STX     → DLE STX      (se escapa)
  C       → C
  ETX     → DLE ETX      (se escapa)

Trama completa:
[STX] A B DLE DLE DLE STX C DLE ETX [ETX]
 ↑                                   ↑
delimit.                           delimit.

El receptor, al ver DLE, come el DLE y emite el siguiente byte como dato.
Un "DLE DLE" → un solo DLE. Un "DLE STX" → un solo STX.
```

La comparación que cierra la respuesta y es lo que el corrector busca:

	Orientado a **bit**	Orientado a **byte**
Unidad de delimitado	**1 patrón de bits** (`01111110`)	**1 byte** (`DLE` , `0x7E`)
Mecanismo	**bit stuffing**: un `0` cada 5 unos	**byte stuffing**: un byte ESC antes de cada byte peligroso
Robustez ante desalineación	**Sí**: se engancha bit a bit	**No**: si se pierde un bit, deja de ver bytes
Overhead	**bajo** (solo en las rachas largas)	**mayor** (2 bytes por byte escapado)
Cuándo se usa	medios que pueden desalinear bits (HDLC, enlaces síncronos)	medios byte-a-byte (PPP, POP3, SDLC)

> **Tope para 10 puntos:** la definición (2), la necesidad + sincronización (1), **bit stuffing con el flag `01111110` y el ejemplo** (3), **byte stuffing con el escape y el ejemplo** (3), y la distinción de cuándo se usa cada uno (1).

3.2 Examen 2010-03, Pregunta 3 (8 puntos) — PPP

> **Fuente:** Examen `eirc1003` / `sirc1003` — marzo de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf

Enunciado (texto del examen)

> Pregunta 3 (8 puntos)
>
> a) ¿Qué es el protocolo PPP? ¿Qué es lo que lo hace más simple que otros protocolos que se ejecutan en la misma capa como por ejemplo CSMA?
>
> b) PPP utiliza flags para delimitar sus mensajes. Indique cómo es el mecanismo por el cual se logra distinguir entre el contenido de los mensajes y las flags que lo delimitan.

Respuesta oficial

> **a)** PPP (point-to-point protocol) es un protocolo de capa de enlace que se utiliza para enviar y recibir datos entre dos entidades en un enlace punto a punto. Es más simple que los protocolos de acceso al medio de la capa de enlace debido a que no se utiliza en un medio broadcast, donde muchos emisores necesitan competir a la vez por el canal, por lo tanto no se necesita un direccionamiento MAC explícito.
>
> **b)** PPP delimita las tramas usando un byte especial considerado como `<flag>` al inicio y al fin de cada trama. Si este byte aparece en algún punto del contenido del mensaje se realiza byte stuffing del mismo utilizando un byte de `<escape>` : - El emisor agrega un byte `<escape>` antes de cualquier byte `<flag>` o byte `<escape>` que esté en el contenido que debe enviar. - Si el receptor encuentra un byte `<flag>` , considera que es el inicio o el fin de una trama. Si encuentra un byte `<escape>` , lo descarta y considera el siguiente byte como parte del contenido de la trama.

Resolución

(a) Ya está perfecto arriba. Lo que hay que sumar para llegar al tope: **PPP multiplexa** varios protocolos de red (el campo *Protocol* = 2 bytes dice cuál viaja), lo que es imposible sobre un medio broadcast sin una dirección MAC. Y **PPP no implementa CSMA/CA ni CSMA/CD** porque en un enlace punto a punto la **colisión es estructuralmente imposible**: sólo hay dos estaciones y una está en silencio mientras la otra transmite.

(b) Es byte stuffing puro, idéntico a §2.1. Lo que hay que agregar para completar:

```
Números concretos de PPP:
   FLAG = 0x7E  = 01111110
   ESC  = 0x7D  = 01111101
   Escape de 0x7E  →  0x7D 0x5E
   Escape de 0x7D  →  0x7D 0x5D

Y el formato completo:
[0x7E][0xFF][0x03][Protocol:2][Payload:0..1500][FCS:2][0x7E]
0xFF = "no hay dirección" (PPP no direcciona: el enlace es punto a punto)
0x03 = "trama no numerada, sin secuencia, sin ACK"
```

Interpretación. El campo Address fijo en `0xFF` es la demostración en el formato de lo que dice el ítem (a): **no hay direccionamiento porque no hay a quién direccionar**. Sólo dos stations, y ya se sabe quién es quién.

PARTE 4 — CIERRE

📌 Resumen

1. La capa física **no sabe de mensajes**: transmite bits. Sin saber dónde acaba un paquete, el receptor no puede hacer nada con él.
2. **Entramado** = poner marcas para que el receptor ubique los límites de cada mensaje. Su segundo propósito es permitir la **resincronización**.
3. Las **cinco funciones** de la capa 2: entramado, acceso al medio, detección de errores, control de flujo y **transparencia**.
4. **Conteo de bytes**: un solo bit perdido desplaza la lectura de forma **permanente e irrecuperable**. Por eso no se usa solo.
5. **Byte stuffing**: se escapan `FLAG` y `ESC`. El receptor come el ESC y toma el siguiente byte como literal.
6. **Bit stuffing** (HDLC): un `0` después de **cinco** unos seguidos.
7. **Por qué cinco**: el flag `01111110` tiene **seis** unos. Cinco es el mayor stuffing que todavía garantiza que los datos no puedan contener un flag ⇒ el flag es único ⇒ el receptor se resincroniza solo tras cualquier error.
8. **HDLC**: tres tipos de trama (Información, Supervisión, No numerada), con *piggybacking* y ARQ de ventana deslizante.
9. **PPP**: cuatro fases (LCP → autenticación → NCP → datos). Simple porque no hay contención ni direccionamiento en un enlace punto a punto.
10. En 2026 el stuffing es **herencia histórica**; los enlaces de datos usan longitud explícita + CRC fuerte. La teoría **se sigue preguntando**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la capa física no puede, por sí sola, entregar un mensaje?

<details>
<summary>Respuesta</summary>
</details>

2. Enumerá las cinco funciones de la capa de enlace con su propósito.

<details>
<summary>Respuesta</summary>
</details>

3. ¿Cuál es el problema fatal del conteo de bytes? Formulalo en una frase.

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué HDLC inserta un cero después de **cinco** unos seguidos y no después de seis, ni de tres?

<details>
<summary>Respuesta</summary>
</details>

5. Con HDLC, si un receptor de HDLC pierde un bit en medio de la trama, ¿se recupera? Explicá en dos pasos.

▶ Respuesta

6. Hacé el byte stuffing de `7E 41 7D 42 7E` con FLAG = 0x7E y ESC = 0x7D. Decí qué va por el cable.

▶ Respuesta

7. ¿Por qué PPP es más simple que CSMA, según la respuesta oficial?

▶ Respuesta

8. ¿Qué cuatro fases tiene PPP y para qué sirve cada una?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0612` / `sirc0612`** — P2 (10 pts, framing: por qué + bit stuffing + byte stuffing) · P5 (5 pts, estación oculta — clase 43)
- **Examen `eirc1003` / `sirc1003`** — P3 (8 pts, PPP y byte stuffing)
- **Examen `eirc0812` / `sirc0812`** — P4 (7 pts, POP3 y el mecanismo DLE, que es byte stuffing con otro nombre)
- **Examen `eirc0703` / `sirc0703`** — Problema 1 (30 pts, protocolo de capa 2 con ARQ Selective Repeat y *piggybacking* — clase 40)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 3 (capas inferiores) y cap. 4 (capas de enlace)
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach*, §5.1 ("The Link Layer: Five Parties") — el stuffing y HDLC
- **Práctico 5 de 2008** — `material-oficial/practicos/png/prac05_2008-p01.png`. **Ojo: no tiene ejercicio de entramado**; el Ejercicio 12 es de ARP.
- `RFC 826` — ARP (1982): por qué el entramado sigue siendo necesario
- **PPP** — `RFC 1661` (verificar) · PPPoE — `RFC 2516` (verificar)
- **IEEE 802.3** — formato de trama con longitud explícita (sin stuffing) · **IEEE 802.11** — preámbulo, FCS y tramas de gestión sin FCS
- **Clase siguiente:** 39 — Detección de errores: paridad, checksum y CRC